

UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

Subject:	EE-170L Computer Programming Lab
Lab No:	Lab 12
Lab Title:	Pointers in C++
Semester:	Semester 2
Session:	Spring 2026

Submitted By:

Name: Muhammad Adil

Registration No: BF25NWELE0669

1. OBJECTIVE

To learn and apply pointers in C++ including declaration, initialization, referencing, dereferencing, and pointer arithmetic. To understand how pointers store memory addresses and enable efficient array traversal.

2. THEORY

A pointer is a variable that stores the memory address of another variable. Key concepts include: (1) Pointer Declaration: `dataType *pointerName;` - the asterisk indicates a pointer variable. (2) Referencing Operator (&): Retrieves the memory address of a variable. Example: `ptr = #` stores the address of `num` in `ptr`. (3) Dereferencing Operator (*): Accesses the value at the address stored in the pointer. Example: `*ptr = 10;` changes the value at the address to 10. (4) Pointer Arithmetic: Pointers can be incremented/decremented to traverse arrays. `ptr++` moves to the next memory location based on data type size.

Important notes: Pointers hold addresses, not values directly. The `*` operator has two meanings: multiplication (binary) and dereferencing (unary). The `&` operator has two meanings: bitwise AND (binary) and address-of (unary).

3. LAB TASKS

3.1 Pointer Declaration and Initialization

Declare an integer variable called `num` and initialize it with a value of your choice. Declare a pointer variable called `ptr` of type integer. Initialize the pointer variable to point to the address of the `num` variable. Display the value of `num` and the value pointed to by `ptr` using the dereference operator.

Source Code:

```
#include <iostream>
using namespace std;

int main()
{
    // =====
    // Variable and Pointer Declaration
    // =====
    int num = 42;           // Declare and initialize integer variable
    int *ptr;               // Declare integer pointer (stores memory address)

    // =====
    // Pointer Initialization
    // =====
    // & (address-of operator) gets the memory address of num
    ptr = &num;             // ptr now stores the address of num

    // =====
```

```

// Display Information about Variables and Pointers
// =====
cout << "===== POINTER BASICS =====" << endl;
cout << "Value of num:          " << num << endl;
cout << "Address of num (&num):  " << &num << endl;
cout << "Value of ptr:           " << ptr << endl;
cout << "Address of ptr (&ptr):   " << &ptr << endl;

// =====
// Dereferencing Demonstration
// =====
// * (dereference operator) accesses value at stored address
cout << "\nValue pointed by ptr (*ptr): " << *ptr << endl;

// =====
// Modify Value through Pointer
// =====
// Changing value using dereference operator
// This modifies the actual variable num through its pointer
*ptr = 100; // Changes the value at address stored in ptr

cout << "\nAfter *ptr = 100:" << endl;
cout << "Value of num (modified): " << num << endl;
cout << "Value pointed by ptr:     " << *ptr << endl;
cout << "===== " << endl;

return 0;
}

```

Expected Output:

```

===== POINTER BASICS =====
Value of num:          42
Address of num (&num): 0x7ffd1234abcd
Value of ptr:         0x7ffd1234abcd
Address of ptr (&ptr): 0x7ffd1234abcc

Value pointed by ptr (*ptr): 42

After *ptr = 100:
Value of num (modified): 100
Value pointed by ptr:    100
=====

```

3.2 Pointer Arithmetic

Declare an integer array called numbers with some values of your choice. Declare a pointer variable called ptr of type integer and initialize it to point to the first element of the numbers array. Use pointer arithmetic to access and display the elements of the array using the pointer.

Source Code:

```
#include <iostream>
using namespace std;

int main()
{
    // =====
    // Array Declaration and Initialization
    // =====
    int numbers[5] = {10, 20, 30, 40, 50};

    // =====
    // Pointer Declaration and Initialization
    // =====
    // ptr points to the first element of the array
    // Array name itself acts as a pointer to first element
    int *ptr = numbers; // Equivalent to: ptr = &numbers[0]

    // =====
    // Display Array using Array Index Notation
    // =====
    cout << "===== ARRAY USING INDEX NOTATION =====" << endl;
    for (int i = 0; i < 5; i++)
    {
        cout << "numbers[" << i << "] = " << numbers[i] << endl;
    }

    // =====
    // Display Array using Pointer Arithmetic
    // =====
    cout << "\n===== ARRAY USING POINTER ARITHMETIC =====" <<
endl;

    // Method 1: Using pointer offset (ptr + i)
    cout << "\nMethod 1: *(ptr + i) offset notation" << endl;
    for (int i = 0; i < 5; i++)
    {
        cout << "**(ptr + " << i << ") = " << *(ptr + i) << endl;
        // ptr + i moves pointer i positions forward
        // *(ptr + i) dereferences to get the value at that address
    }

    // Method 2: Direct pointer arithmetic demonstration
    cout << "\nMethod 2: Direct dereferencing" << endl;
    cout << "*ptr          = " << *ptr << endl;           // First element (10)
```

```

    cout << "*(ptr + 1) = " << *(ptr + 1) << endl; // Second element (20)
    cout << "*(ptr + 2) = " << *(ptr + 2) << endl; // Third element (30)
    cout << "*(ptr + 3) = " << *(ptr + 3) << endl; // Fourth element (40)
    cout << "*(ptr + 4) = " << *(ptr + 4) << endl; // Fifth element (50)

    // =====
    // Demonstrate Pointer Increment
    // =====
    cout << "\n===== POINTER INCREMENT DEMO =====" << endl;
    int *temp = numbers; // Start pointer at beginning of array

    for (int i = 0; i < 5; i++)
    {
        cout << "Address: " << temp << ", Value: " << *temp << endl;
        temp++; // Move pointer to next integer location
                // Adds sizeof(int) bytes to the address
    }

    cout << "===== " << endl;

    return 0;
}

```

Expected Output:

```

===== ARRAY USING INDEX NOTATION =====
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50

===== ARRAY USING POINTER ARITHMETIC =====

Method 1: *(ptr + i) offset notation
*(ptr + 0) = 10
*(ptr + 1) = 20
*(ptr + 2) = 30
*(ptr + 3) = 40
*(ptr + 4) = 50

Method 2: Direct dereferencing
*ptr      = 10
*(ptr + 1) = 20
*(ptr + 2) = 30
*(ptr + 3) = 40
*(ptr + 4) = 50

```

```

===== POINTER INCREMENT DEMO =====
Address: 0x7ffd1234abc0, Value: 10
Address: 0x7ffd1234abc4, Value: 20
Address: 0x7ffd1234abc8, Value: 30
Address: 0x7ffd1234abcc, Value: 40
Address: 0x7ffd1234abd0, Value: 50
=====

```

3.3 Demonstration: Pointer Operations with Multiple Variables

Demonstrate pointer usage with multiple variables, showing how pointers can modify values and copy data between variables through dereferencing.

Source Code:

```

#include <iostream>
using namespace std;

int main()
{
    // =====
    // Variable Declaration
    // =====
    int a = 5;        // First integer variable
    int b = 15;       // Second integer variable

    // =====
    // Pointer Declaration
    // =====
    int *p1, *p2;     // Declare two integer pointers

    // =====
    // Initialize Pointers to Point to Variables
    // =====
    p1 = &a;          // p1 points to variable a (stores address of a)
    p2 = &b;          // p2 points to variable b (stores address of b)

    // =====
    // Display Initial Values
    // =====
    cout << "===== POINTER OPERATIONS =====" << endl;
    cout << "Initial values:" << endl;
    cout << "a = " << a << ", b = " << b << endl;
    cout << "*p1 = " << *p1 << ", *p2 = " << *p2 << endl;

    // =====
    // Modify Values through Pointers
    // =====
    *p1 = 10;         // Change value of a through pointer p1

```

```

cout << "\nAfter *p1 = 10:" << endl;
cout << "a = " << a << ", *p1 = " << *p1 << endl;

// =====
// Copy Value through Pointers
// =====
*p2 = *p1;          // Copy value pointed by p1 to location pointed by p2
cout << "\nAfter *p2 = *p1:" << endl;
cout << "b = " << b << ", *p2 = " << *p2 << endl;

// =====
// Change Value Again through Pointer
// =====
*p1 = 20;           // Change value of a again
cout << "\nAfter *p1 = 20:" << endl;
cout << "a = " << a << ", b = " << b << endl;
cout << "*p1 = " << *p1 << ", *p2 = " << *p2 << endl;

cout << "===== " << endl;

return 0;
}

```

Expected Output:

```

===== POINTER OPERATIONS =====
Initial values:
a = 5, b = 15
*p1 = 5, *p2 = 15

After *p1 = 10:
a = 10, *p1 = 10

After *p2 = *p1:
b = 10, *p2 = 10

After *p1 = 20:
a = 20, b = 10
*p1 = 20, *p2 = 10
=====

```

4. CONCLUSION

This lab demonstrated the fundamental concepts of pointers in C++. We learned that pointers store memory addresses rather than values, and how to use the referencing (&) and dereferencing (*) operators. Pointer arithmetic allows efficient traversal of arrays without using index notation. Pointers are powerful tools that enable dynamic memory allocation, efficient array handling,

function parameter passing by reference, and complex data structures. Understanding pointers is essential for advanced C++ programming and systems-level development.